CS6650 Assignment 2 Report

- CS6650 Assignment 2 Report
  - 1. Git Repository URL
  - 2. Architecture Document (Front Part)
    - 2.1 System Architecture
    - 2.2 Message Flow Sequence
    - 2.3 Queue Topology Design
    - 2.4 Consumer Threading Model
    - 2.4.2 Client Thread Model Decision (Why no 128+ in final runs)
    - 2.4.1 Inflight Strategy (Why `inflight=8`, and why keep `inflight=1`)
    - 2.5 Load Balancing Configuration
    - 2.6 Failure Handling Strategies
    - 2.7 Multi-Consumer Scaling Plan and Terraform Impact
  - 3. Test Results
    - 3.1 Metrics Requirements Checklist (Assignment Alignment)
    - 3.2 Test Matrix and Summary Table
    - 3.3 Screenshot Checklist
    - 3.4 Configuration Snapshot Per Run
    - 3.5 Queue Profile Assessment
    - 3.6 System Utilization Snapshot (Minimal Overhead Approach)
    - 3.7 Improvement Analysis
    - 3.8 Post-Assignment Architecture Direction

CS6650 Assignment 2 Report

- Course: CS6650 Building Scalable Distributed Systems
- Assignment: Assignment 2 - Adding Message Distribution and Queue Management
- Date: 2026-03-12
- Repository URL: `<https://github.com/Eternity1824/chatflow.git>`

1. Git Repository URL

`<https://github.com/Eternity1824/chatflow.git>`

The repository includes the required folders: - `/server-v2` - `/consumer` - `/deployment` - `/monitoring`

2. Architecture Document (Front Part)

2.1 System Architecture

The deployed system separates concerns into ingress, queueing, and fan-out layers:

1. Clients connect via WebSocket to AWS ALB.
2. ALB forwards/sticks each connection to one `server-v2` instance.
3. `server-v2` validates incoming chat messages and publishes them to RabbitMQ.
4. RabbitMQ routes messages by room (`room.{roomId}`).
5. `consumer` instances (room-sharded) read assigned room queues in push mode, process protobuf payloads, and call each server's internal gRPC broadcast service.
6. Each `server-v2` instance broadcasts to its local in-memory room sessions.

```
Client Threads
    |
    v
AWS ALB (sticky sessions)
    |
    +----------------------+----------------------+----------------------+
    v                      v                      v
server-v2 #1           server-v2 #2          server-v2 #N
(validate + publish)   (validate + publish)  (validate + publish)
    \                      |                      /
     \_____________________|_____________________/
                           |
                           v
          RabbitMQ topic exchange: chat.exchange
                           |
          routing key: room.{roomId}
                           |
      +----------------+----------------+
      v                                 v
  queue room.1 ...                    queue room.20
      |                                 |
      +------------ room-sharded -------+
                    assignment
                      |
                      v
      +----------------------------------------------+
      | Consumer Instance #1 (worker pool)          |
      | Consumer Instance #2 (worker pool)          |
      | ...                                         |
      | Consumer Instance #M (worker pool)          |
      +----------------------------------------------+
                      |
                      v
      gRPC InternalBroadcast.Broadcast (fanout to all servers)
                      |
                      v
        Each server does local room broadcast to in-memory sessions
```

2.2 Message Flow Sequence

A. Producer path (client -> queue)

1. Client sends `JOIN/TEXT/LEAVE` to `/chat?roomId=<room>`.
2. `RoomIdExtractorHandler` validates room ID and serves `/health` checks.
3. `WebSocketChatHandlerV2` parses JSON, validates fields, enforces `JOIN` before `TEXT`.
4. Server creates `QueueChatMessage` with:
   - `messageId` (UUID)
   - `roomId`, `userId`, `username`, `message`, `timestamp`, `messageType`
   - `serverId`, `clientIp`
5. `RabbitMqPublisher` converts message to protobuf bytes and publishes to `chat.exchange` with routing key `room.{roomId}`.
6. Publisher confirm is handled with async confirm tracking (configurable sync wait can still be enabled).

B. Consumer path (queue -> client broadcast)

1. `ProtobufConsumerWorker` subscribes queues with `basicConsume(..., autoAck=false)` and per-worker `basicQos(prefetch)`.
2. Payload is parsed as protobuf; malformed payloads are ACKed and dropped.
3. Dedup check (`messageId`) avoids duplicate side effects.
4. If missing, room sequence is assigned by `RoomSequenceManager`.
5. Delivery enters per-room pending queue; worker dispatches when `roomMaxInFlight` and `globalMaxInFlight` allow.
6. `GrpcBroadcastClient` sends protobuf message to each server gRPC endpoint.
7. On success: queue message is ACKed.
8. On failure: retry with exponential backoff + jitter; if retries exhausted, `NACK(requeue=false)`.

2.3 Queue Topology Design

- Broker: RabbitMQ (durable topic exchange).
- Exchange: `chat.exchange`.
- Routing key convention: `room.{roomId}`.
- Queue model: one queue per room, default range `room.1` to `room.20`.
- Queue durability: enabled.
- Queue controls:
  - `x-message-ttl` default `60000 ms`
  - `x-max-length` default `10000`
- Publisher reliability:
  - persistent message properties

    - mandatory publish flag
    - publisher confirms

This topology isolates room traffic, keeps ordering naturally scoped per queue, and allows independent tuning via queue and consumer parameters.

2.4 Consumer Threading Model

- Entry point: `ConsumerApp`.
- Threads: configurable fixed-size worker pool (`CHATFLOW_CONSUMER_THREADS`).
- Queue assignment: round-robin fair split across workers (`RoomAssignment.assignQueues`).
- Worker model:
    - one RabbitMQ connection + channel per worker
    - configurable prefetch (`basicQos`)
    - event-driven callback (`basicConsume`) on Netty EventLoop
    - per-room pending queue + inflight gating
- Shared state (thread-safe):
    - dedup cache (`MessageDeduplicator`)
    - per-room sequence counters (`RoomSequenceManager`)
    - metrics counters (`ConsumerMetrics`)
- Health/observability:
    - `/health` returns `OK`
    - `/metrics` exports JSON counters

Note: scaling is configuration-driven (threads + instance count + room shard assignment), not full autoscaling yet.

2.4.2 Client Thread Model Decision (Why no 128+ in final runs)

For this implementation, client send path is Netty event-loop + non-blocking write/flush with a global rate limiter. Under this model, increasing application sender threads beyond a moderate value did not improve throughput and increased instability.

Observed on March 11, 2026 (single server + single consumer, targetQps=4500):

- 32 threads: ~4366 msg/s, lower error ratio, better E2E p99
- 64 threads: ~4311 msg/s, higher error ratio than 32
- 128 threads: ~4224 msg/s, highest error ratio and worse tail latency

Conclusion: - final load tests use **32 sender threads** as default profile - 64-thread runs are kept only as reference, because on this Netty event-loop path they add context-switch overhead and higher tail instability without throughput gain - exclude 128/256/512 from final optimization set because extra sender threads add context-switch overhead without throughput gain in this Netty-based path

Netty-specific rationale: - effective concurrency comes from event loops + non-blocking flush, not from continuously increasing application sender threads - once event loops are saturated, raising sender thread count mostly increases scheduling/queue contention instead of useful I/O work

2.4.1 Inflight Strategy (Why `inflight=8`, and why keep `inflight=1`)

We use two operation modes intentionally:

- Throughput mode (`CHATFLOW_ROOM_MAX_INFLIGHT=8`):
    - goal: maximize queue drain rate and overall throughput
    - tradeoff: room-local delivery order may be relaxed under bursty load
    - mitigation: include room sequence in payload and let client reorder by sequence when needed
- Strict-order mode (`CHATFLOW_ROOM_MAX_INFLIGHT=1`):
    - goal: preserve room-local processing order in consumer path
    - tradeoff: lower max throughput due to serialized per-room dispatch
    - use: validation runs and requirement-focused evidence

This dual-mode strategy keeps one profile for peak performance tuning and one profile for strict room-order guarantee.

2.5 Load Balancing Configuration

Infrastructure is provisioned with Terraform (`deployment/terraform`):

- ALB listener: HTTP `:80` -> server target group.
- Target group protocol/port: HTTP on `chat_port` (default `8080`).
- Sticky sessions: enabled (`lb_cookie`, default `3600s`).
- Idle timeout: default `120s` (satisfies WebSocket > 60s requirement).
- Health check:
    - path `/health`
    - interval `30s`
    - timeout `5s`
    - healthy threshold `2`
    - unhealthy threshold `3`
- Server fleet size is configurable (`server_count`) for 1/2/4-instance test scenarios.

2.6 Failure Handling Strategies

1. Queue publish protection (server side)

- Channel pooling reduces connection churn.
- Circuit breaker opens after consecutive publish failures and rejects requests temporarily.
- Half-open probe allows controlled recovery.

2. Consumer delivery resilience

- At-least-once semantics with manual ACK.
- Retry with bounded exponential backoff and jitter.
- Retry count propagated via message header (`x-chatflow-retry`).

3. Duplicate and ordering controls

- Dedup in consumer (`MessageDeduplicator`) and internal broadcast path (`RecentMessageTracker`).
- Room-local sequencing via `RoomSequenceManager` when sequence is absent.

4. Operational safety

- Health endpoints on server and consumer.
- Periodic metrics logging and `/metrics` endpoint for quick diagnosis.
- Graceful shutdown hooks close workers and servers.

2.7 Multi-Consumer Scaling Plan and Terraform Impact

Given the fixed room range (`1..20`), horizontal scaling is done by room sharding instead of random queue competition.

Recommended shard rule: - `owner = (roomId - 1) % consumer_instance_count` - consumer instance `i` only consumes rooms where `owner == i`

Benefits: - predictable load split - better control of room-local ordering semantics - simpler bottleneck analysis per shard

Terraform/deployment impact: - add `consumer_count` (number of consumer EC2 instances) - pass shard env vars per consumer instance: - `CHATFLOW_CONSUMER_INSTANCE_INDEX` - `CHATFLOW_CONSUMER_INSTANCE_COUNT` - for gRPC internal fan-out, ensure security group allows `consumer -> server:9090`

For assignment runs, we plan: - 1 server scenario -> 1 consumer shard - 2 server scenario -> 2 consumer shards - 4 server scenario -> 4 consumer shards

3. Test Results

3.1 Metrics Requirements Checklist (Assignment Alignment)

For each run, ensure the following metrics are captured:

- Client metrics:
  - Total runtime
  - Throughput (messages/second)
  - Connection failures
  - Retry attempts
- Queue metrics:
  - Peak queue depth
  - Average queue depth
  - Consumer rate
  - Producer rate
- System metrics:
  - CPU usage (all instances)
  - Memory usage
  - Network I/O
  - Disk I/O

Additional (recommended) metrics used in this report:

- ACK latency (`mean/median/p95/p99`)
- Broadcast E2E latency (`mean/median/p95/p99`)
- Error responses and parse errors

3.2 Test Matrix and Summary Table

Scenario	Server Count	Consumer Count	Message Count	Client Threads	Throughput (msg/s)	ACK p95 (ms)	ACK p99 (ms)	E2E p95 (ms)	E2E p99 (ms)	Conn Failures	Retry Attempts	Peak Queue Ready	Avg Queue Ready	Notes
Single-server baseline (completed)	1	1	500,000	32	1933.92	17	31	35	251	0	0	~0 (mostly zero)	~0 (mostly zero)	Stable in 1C1S with low ACK/E2E latency and

Scenario	Server Count	Consumer Count	Message Count	Client Threads	Throughput (msg/s)	ACK p95 (ms)	ACK p99 (ms)	E2E p95 (ms)	E2E p99 (ms)	Conn Failures	Retry Attempts	Peak Queue Ready	Avg Queue Ready	Notes
														no queue backlog
Load-balanced (2 servers, completed)	2	2	500,000	32	7738.12	44	240	2855	3689	0	0	0	~0	High ingress achieved, but RabbitMQ (`t3.small`) became the bottleneck at higher QPS with visible spike behavior
Load-balanced (4 servers, completed)	4	4	500,000	32	4844.71	37	52	100	820	0	0	0	~0	Stable low queue-ready depth; RabbitMQ CPU saturates on `t3.small` and limits further scale-out gain

3.3 Screenshot Checklist

A. Single Instance Scenario

- [x] Client terminal output screenshot (performance summary)
- [x] RabbitMQ Overview screenshot (queue depth + message rates)
- [x] RabbitMQ Queues screenshot (all room queues with ready/unacked/rates)
- [x] EC2 system metrics screenshot(s): CPU/Network/Disk
- [x] Memory evidence screenshot (`free -m` or `top`)

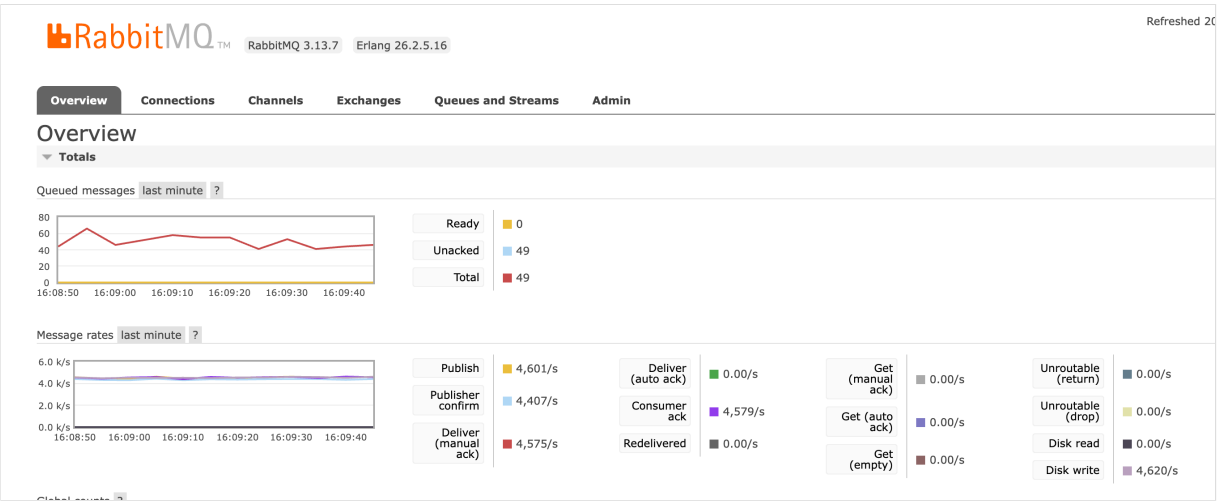
Single-instance (1C1S) evidence has been collected under `results/pics`:

Client Output (`metrics-1s1c.png`)

```
=== Performance Metrics ===
Successful messages: 488829
Error responses: 11171
Broadcast deliveries: 461177
Out-of-order broadcasts: 99269
Connection failures: 0
Retry attempts: 0
Parse errors: 0
Failed messages: 0
Total runtime: 111969 ms
Throughput: 4365.75 messages/second
Total connections: 20
Reconnections: 0
Latency (ms) mean: 29.29
Latency (ms) median: 29
Latency (ms) p95: 36
Latency (ms) p99: 50
Latency (ms) min/max: 15/159
Broadcast E2E Latency (ms) mean: 44.19
Broadcast E2E Latency (ms) median: 38
Broadcast E2E Latency (ms) p95: 61
Broadcast E2E Latency (ms) p99: 273
Broadcast E2E Latency (ms) min/max: 17/498
```

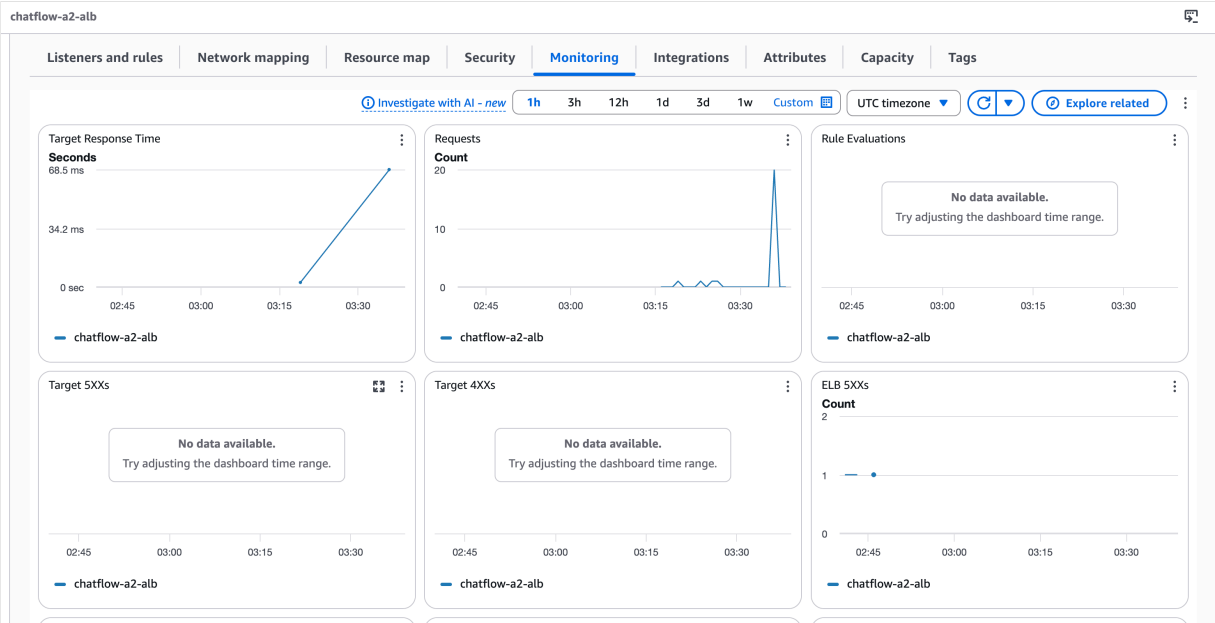
1C1S Client Metrics

RabbitMQ Queues (`rabbitmq-1s1c-32.png`)



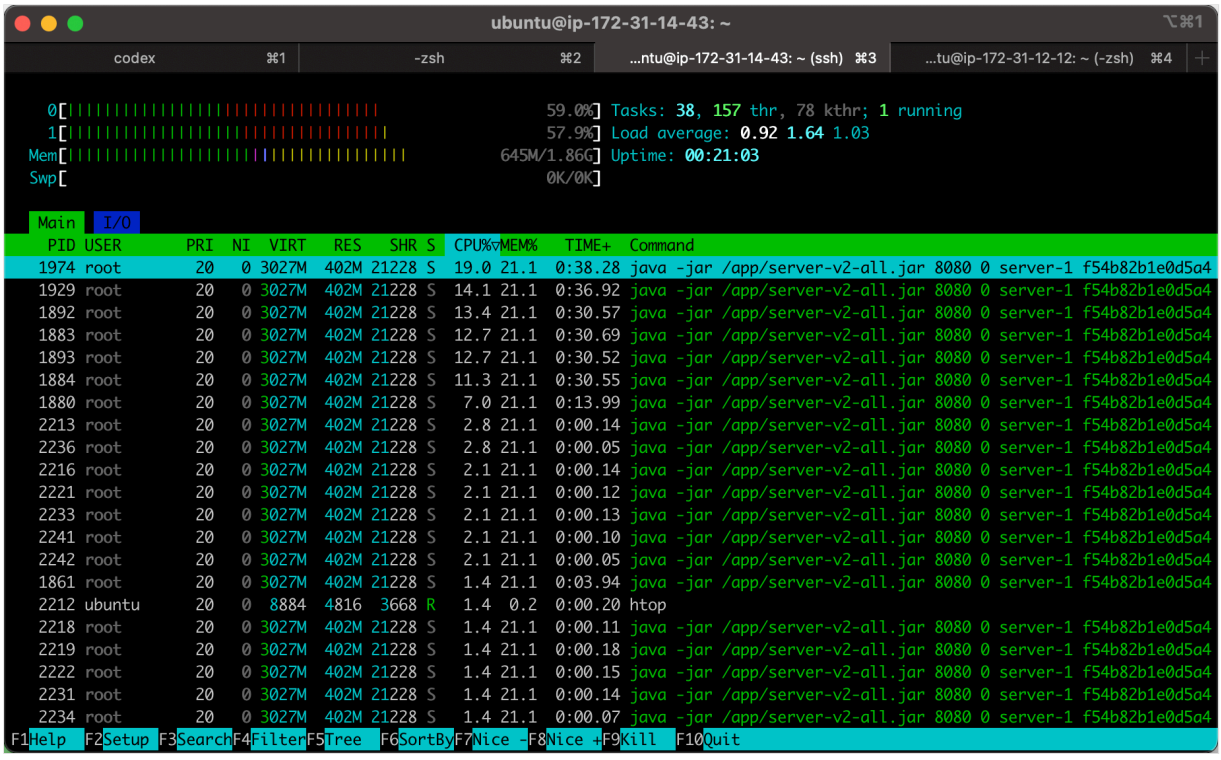
1C1S RabbitMQ Queues

ALB Monitoring (`alb-1c1s.png`)



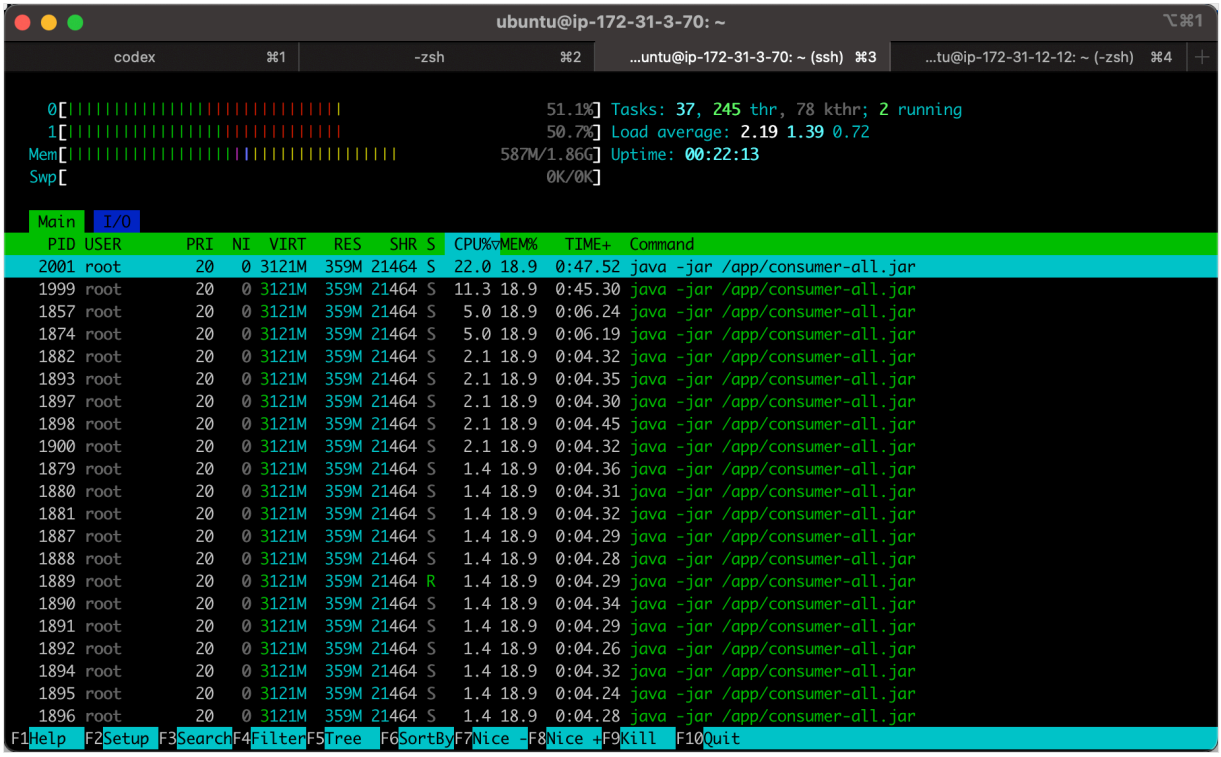
1C1S ALB Monitoring

Server System Snapshot (`server-1s1c-htop.png`)



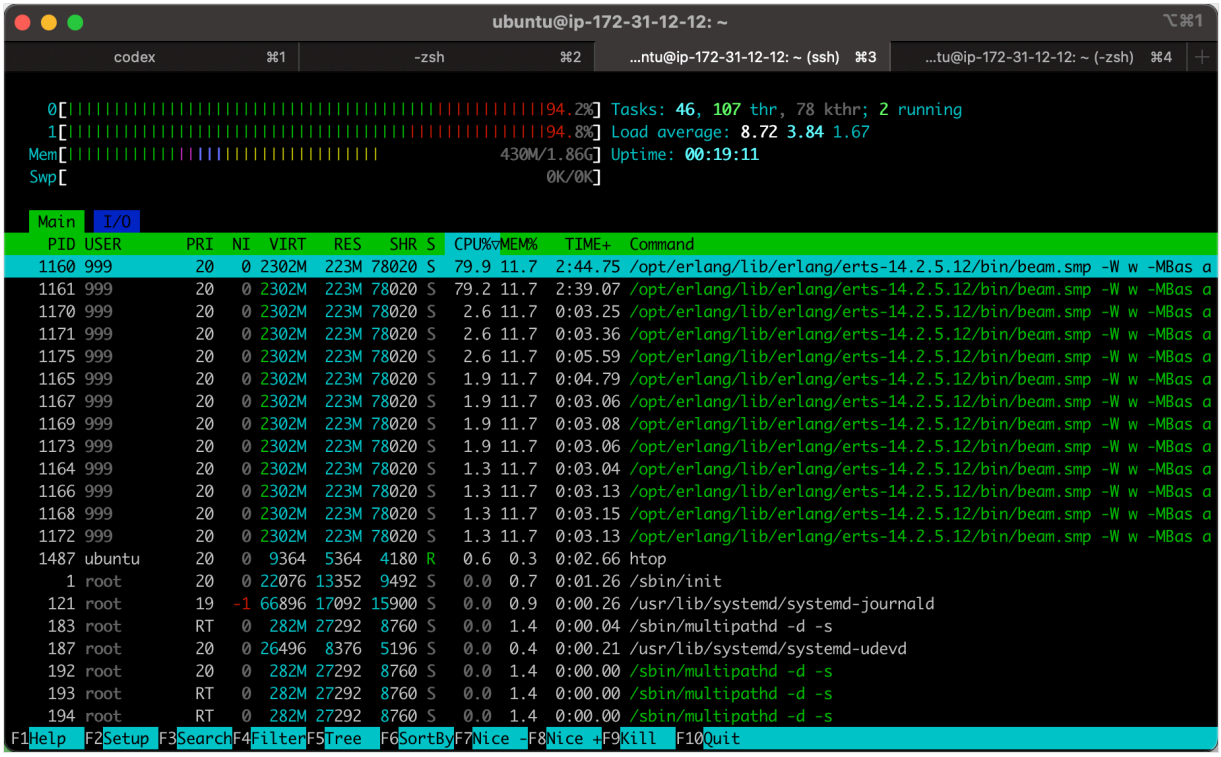
1C1S Server System Snapshot

Consumer System Snapshot (`consumer-1s1c.png`)



1C1S Consumer System Snapshot

RabbitMQ Node Snapshot (`rabbitmq-1s1c-htop.png`)



1C1S RabbitMQ Node Snapshot

B. 2-Instance Load Balanced Scenario

- [x] Client terminal output screenshot
- [x] ALB CloudWatch screenshot (RequestCount, TargetResponseTime, HealthyHostCount)
- [x] RabbitMQ Overview screenshot
- [x] RabbitMQ Queues screenshot
- [x] EC2 system metrics screenshot(s)

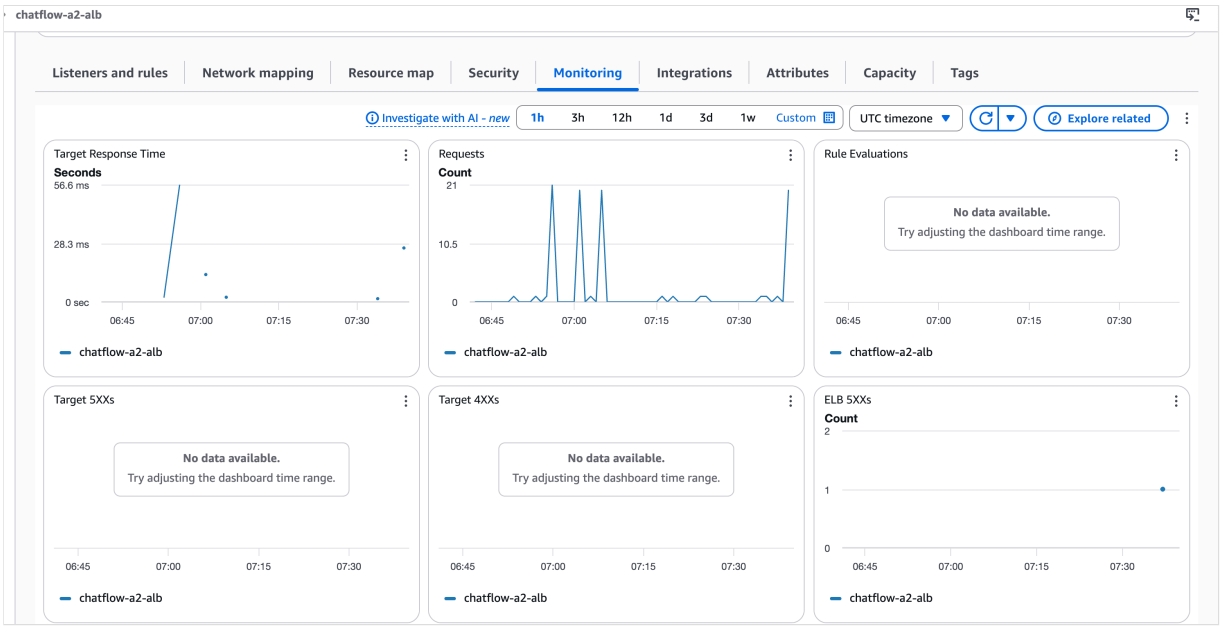
Load-balanced (2C2S) evidence has been collected under `results/pics/2c2s`:

Client Output (`metrics-2c2s.png`)

```
=== Performance Metrics ===
Successful messages: 489312
Error responses: 10688
Broadcast deliveries: 448469
Out-of-order broadcasts: 152865
Connection failures: 0
Retry attempts: 0
Parse errors: 0
Failed messages: 0
Total runtime: 63234 ms
Throughput: 7738.12 messages/second
Total connections: 20
Reconnections: 0
Latency (ms) mean: 36.80
Latency (ms) median: 29
Latency (ms) p95: 44
Latency (ms) p99: 240
Latency (ms) min/max: 16/1361
Broadcast E2E Latency (ms) mean: 741.54
Broadcast E2E Latency (ms) median: 240
Broadcast E2E Latency (ms) p95: 2855
Broadcast E2E Latency (ms) p99: 3689
Broadcast E2E Latency (ms) min/max: 19/4536
```

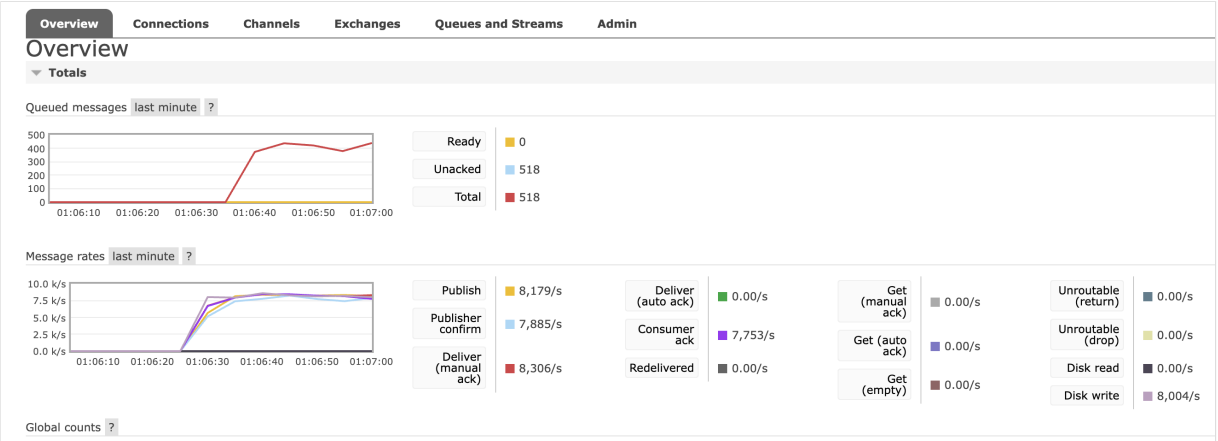
2C2S Client Metrics

ALB Monitoring (`alb-2c2s.png`)



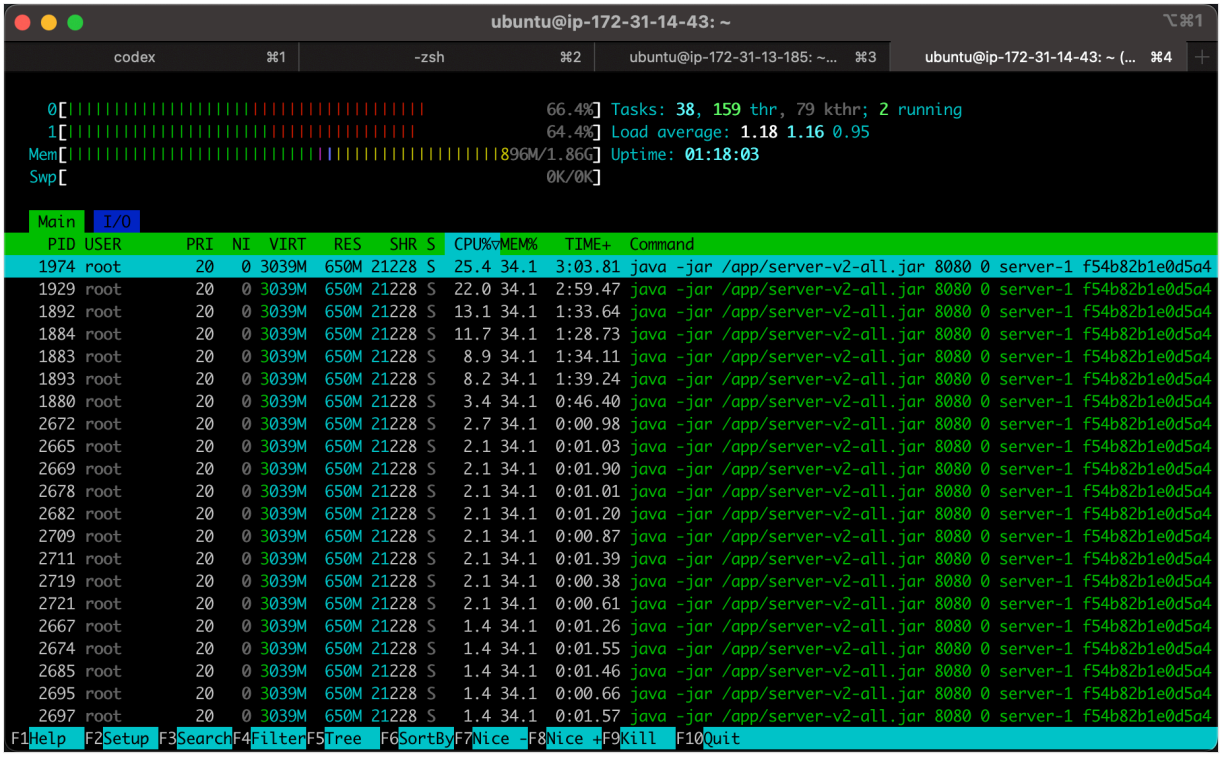
2C2S ALB Monitoring

RabbitMQ Console (`rabbitmq-2c2s.png`)



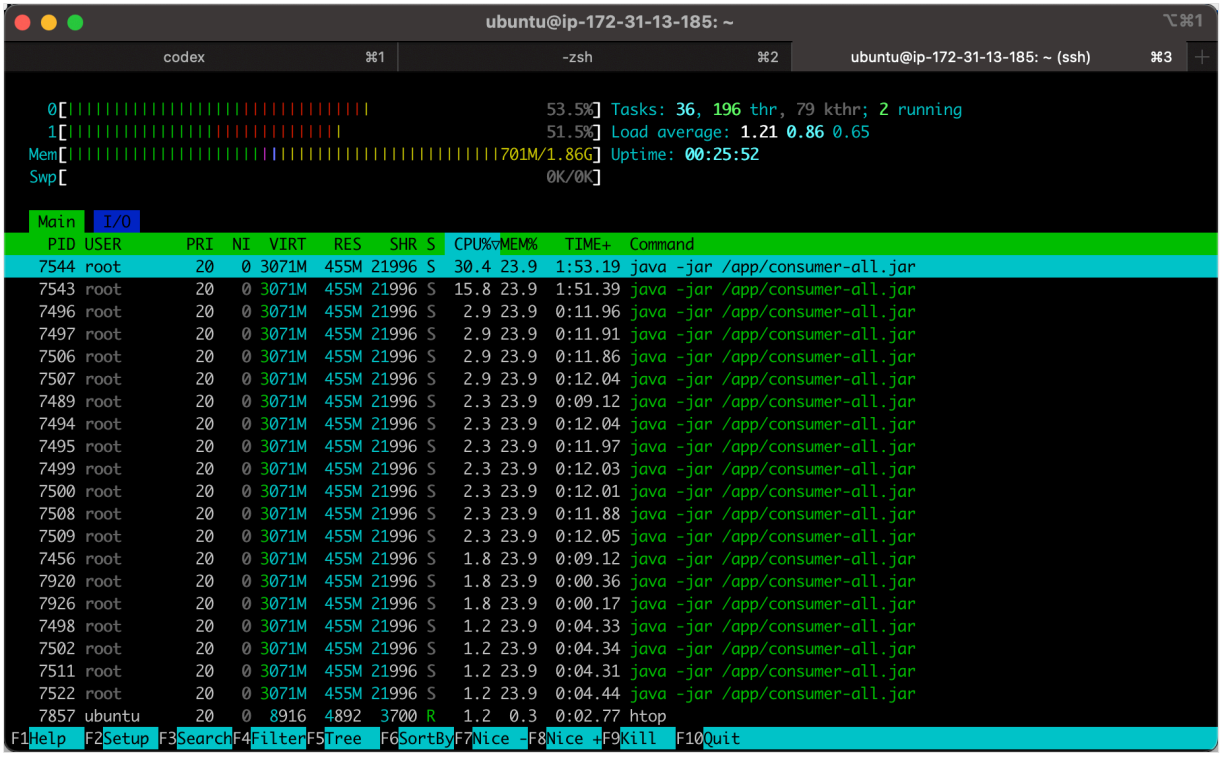
2C2S RabbitMQ

Server System Snapshot (`server-2c2s-htop.png`)



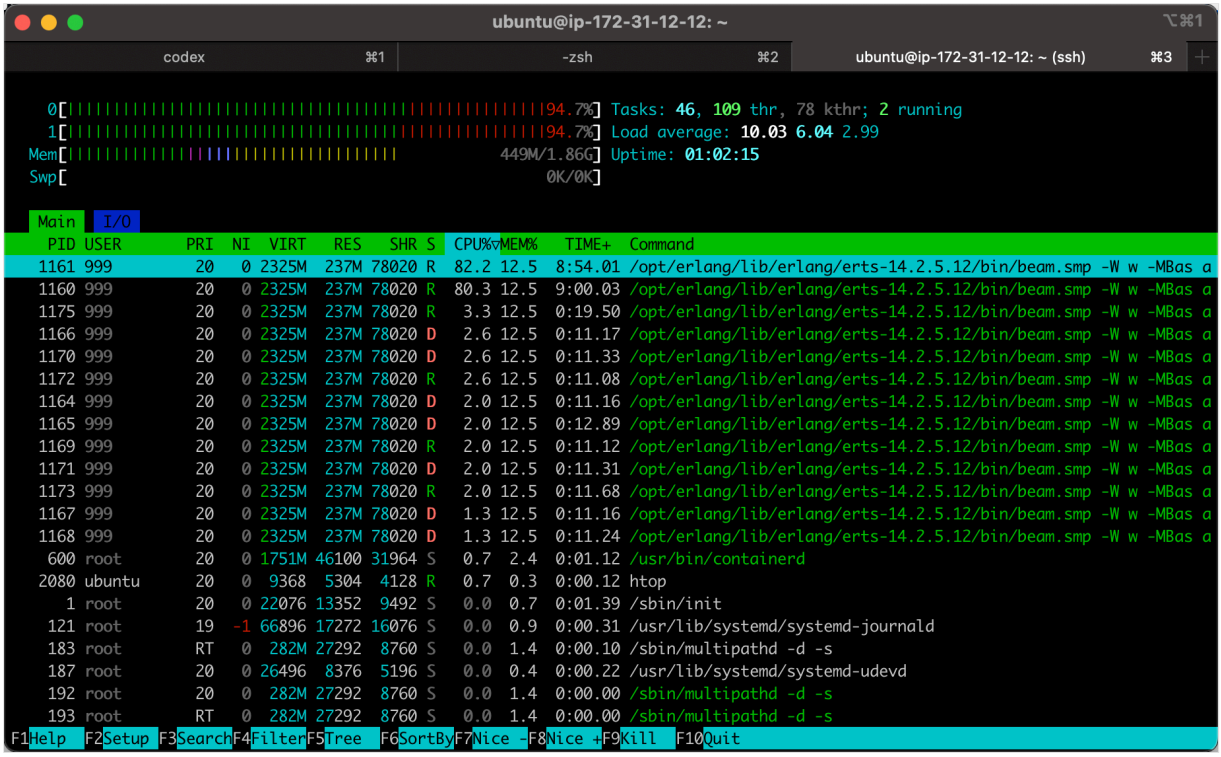
2C2S Server System Snapshot

Consumer System Snapshot (`consumer-2c2s-htop.png`)



2C2S Consumer System Snapshot

RabbitMQ Node Snapshot (`rabbitmq-2c2s-htop.png`)



2C2S RabbitMQ Node Snapshot

C. 4-Instance Load Balanced Scenario

- [x] Client terminal output screenshot
- [x] ALB CloudWatch screenshot (distribution and latency)
- [x] RabbitMQ Overview screenshot
- [x] RabbitMQ queue/rate evidence screenshot
- [x] EC2 system metrics screenshot(s)
- [ ] Optional stress run screenshot (1M messages)

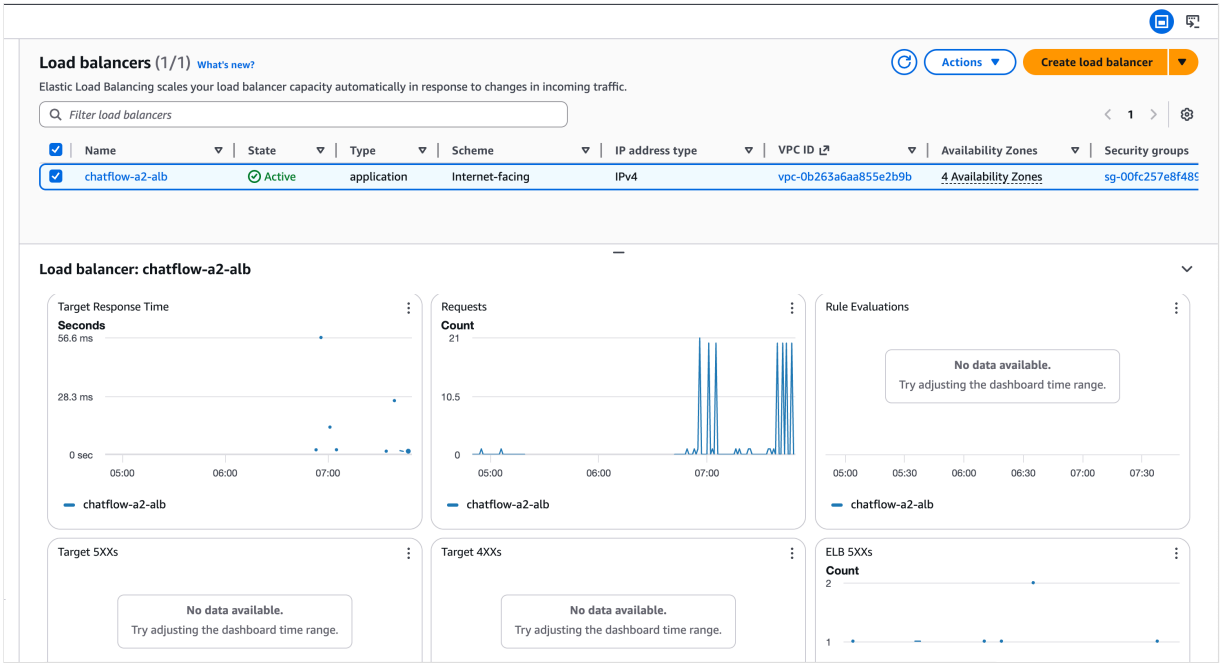
Load-balanced (4C4S) evidence has been collected under `results/pics/4c4s`:

Client Output (`metrics-4c4s.png`)

```
=== Performance Metrics ===
Successful messages: 488880
Error responses: 11120
Broadcast deliveries: 460225
Out-of-order broadcasts: 125030
Connection failures: 0
Retry attempts: 0
Parse errors: 0
Failed messages: 0
Total runtime: 100910 ms
Throughput: 4844.71 messages/second
Total connections: 20
Reconnections: 0
Latency (ms) mean: 30.17
Latency (ms) median: 29
Latency (ms) p95: 37
Latency (ms) p99: 52
Latency (ms) min/max: 15/634
Broadcast E2E Latency (ms) mean: 72.06
Broadcast E2E Latency (ms) median: 48
Broadcast E2E Latency (ms) p95: 100
Broadcast E2E Latency (ms) p99: 820
Broadcast E2E Latency (ms) min/max: 19/1393
```

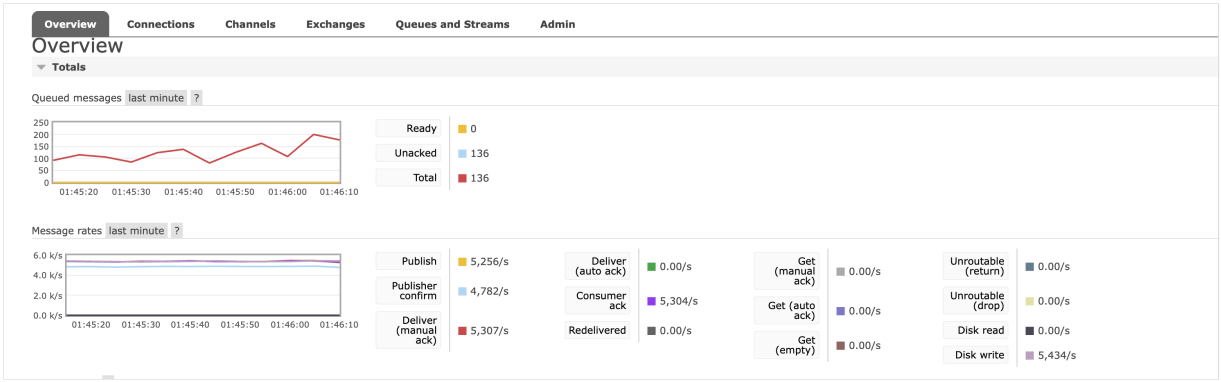
4C4S Client Metrics

ALB Monitoring (`alb-4c4s.png`)



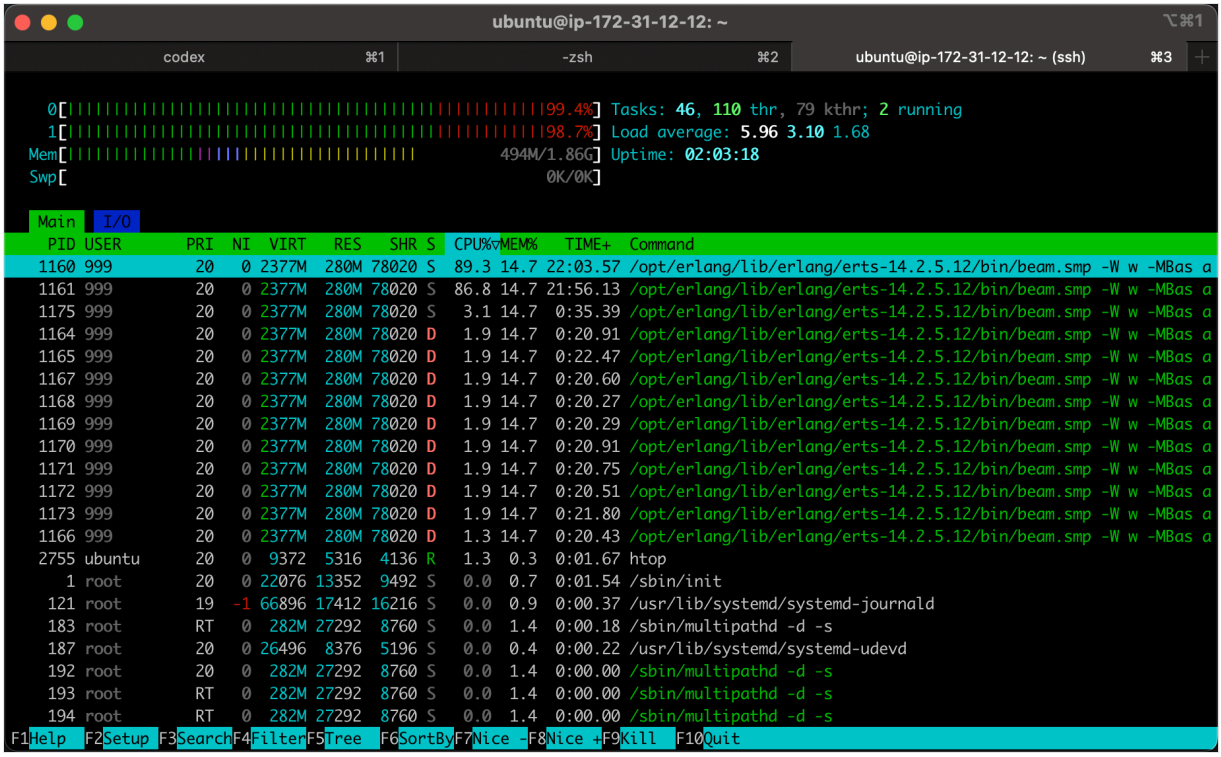
4C4S ALB Monitoring

RabbitMQ Overview (`mq-4c4s.png`)



4C4S RabbitMQ Overview

RabbitMQ Node Snapshot (`mq-4c4s-htop.png`)



4C4S RabbitMQ Node Snapshot

3.4 Configuration Snapshot Per Run

Current filled run (1C1S):

- Run ID: `1c1s-32t-500k`
- Date/Time (PST): `2026-03-11`
- Git commit: `a7542fd (with local run-time config/report edits)`
- Server image tag: `ghcr.io/eternity1824/chatflow-server-v2:v5`
- Consumer image tag: `ghcr.io/eternity1824/chatflow-consumer:v5`
- `server_count: 1`
- `consumer_count: 1`
- `consumer_threads: 80`
- `consumer_prefetch: 200`
- `room_max_inflight: 16`
- `global_max_inflight: 2000`
- Client config file: `config/client-part2-32-s1-4500.yml`

Current filled run (2C2S):

- Run ID: `2c2s-32t-8000-500k`
- Date/Time (PST): `2026-03-12`
- Git commit: `a7542fd (with local run-time config/report edits)`
- Server image tag: `ghcr.io/eternity1824/chatflow-server-v2:v5`
- Consumer image tag: `ghcr.io/eternity1824/chatflow-consumer:v5`
- `server_count: 2`
- `consumer_count: 2`
- `consumer_threads: 40`
- `consumer_prefetch: 160`
- `room_max_inflight: 12`
- `global_max_inflight: 1000`
- Client config file: `config/client-part2-32-s2-8000.yml`

Current filled run (4C4S):

- Run ID: `4c4s-32t-5000-500k`
- Date/Time (PST): `2026-03-12`
- Git commit: `a7542fd (with local run-time config/report edits)`
- Server image tag: `ghcr.io/eternity1824/chatflow-server-v2:v5`
- Consumer image tag: `ghcr.io/eternity1824/chatflow-consumer:v5`
- `server_count: 4`
- `consumer_count: 4`
- `consumer_threads: 80`
- `consumer_prefetch: 200`

- `room_max_inflight`: `16`
- `global_max_inflight`: `2000`
- Client config file: `config/client-part2-32-s4-5000.yml`

3.5 Queue Profile Assessment

For each run, classify queue profile and explain:

Current 1C1S assessment:

- Profile type: `Stable plateau` (publish/ack rates close; queue ready near zero)
- Peak ready depth: `~0` (no meaningful backlog in captured window)
- Drain behavior: `Producer and consumer rates stay close; queue drains in-window`
- Redelivery/duplicate behavior: `No major redelivery spike observed in screenshots`
- Bottleneck hypothesis: `In 1C1S, system is network/ingress limited before queue backlog builds`

Current 2C2S assessment:

- Profile type: `Mostly stable with spike episodes`
- Peak ready depth: `0` (ready queue stayed near zero in captured window)
- Drain behavior: `Consumer ACK rate tracks publish closely, but Unacked/total bursts appear under high ingress`
- Redelivery/duplicate behavior: `No significant redelivery spike shown in RabbitMQ overview screenshot`
- Bottleneck hypothesis: `RabbitMQ node capacity (t3.small) is the primary bottleneck near 8k QPS target`

Current 4C4S assessment:

- Profile type: `Stable plateau with broker CPU pressure`
- Peak ready depth: `0` (ready remained at zero in captured window)
- Drain behavior: `Publish/consumer-ack stayed close around ~5.3k/s, with low queue-ready but non-zero unacked`
- Redelivery/duplicate behavior: `No obvious redelivery spike on overview chart`
- Bottleneck hypothesis: `RabbitMQ on t3.small saturates CPU (~99%), limiting benefit from additional server/consumer instances`

3.6 System Utilization Snapshot (Minimal Overhead Approach)

Prometheus is not required for this assignment.
Use the following low-overhead evidence:

- CloudWatch:
  - ALB: RequestCount, TargetResponseTime, HealthyHostCount
  - EC2: CPUUtilization, NetworkIn/Out, DiskReadBytes/DiskWriteBytes
- On-instance commands (during run):
  - `free -m`
  - `top -b -n 1` (or macOS equivalent when local)
  - optional: `iostat -x 1 5`, `sar -n DEV 1 5`

3.7 Improvement Analysis

- Throughput improvement from 1 -> 2 -> 4 servers:
  - 1C1S: `1933.92 msg/s`
  - 2C2S: `7738.12 msg/s` (about `4.00`x vs 1C1S)
  - 4C4S: `4844.71 msg/s` (about `0.63`x vs 2C2S, constrained by broker)
- Latency trend as scale increases:
  - ACK p95: `17 -> 44 -> 37 ms`
  - ACK p99: `31 -> 240 -> 52 ms`
  - E2E p95: `35 -> 2855 -> 100 ms`
  - E2E p99: `251 -> 3689 -> 820 ms`
  - Note: 2C2S run used higher targetQps (8000), while 4C4S run used targetQps 5000.
- Queue depth trend as scale increases:
  - Ready depth stayed near zero in all captured windows.
  - At higher ingress, unacked and queue total fluctuate, reflecting transient in-flight pressure rather than persistent backlog.
- Resource bottleneck shifts (server vs consumer vs rabbit):
  - 1C1S: no persistent queue bottleneck.
  - 2C2S: RabbitMQ starts to dominate under high ingress.
  - 4C4S: RabbitMQ is clearly the limiting component (CPU saturation near 99% in `mq-4c4s-htop.png`).
- Final tuned configuration and rationale:
  - client main threads: `32` (best stability/throughput tradeoff for this Netty event-loop client path)
  - consumer mode: push (`basicConsume`) with tuned prefetch and async gRPC broadcast
  - room/global inflight: tuned by scenario for throughput while preserving practical room behavior
  - deployment for assignment evidence: 1C1S, 2C2S, 4C4S completed with screenshots and metrics

Preliminary bottleneck note (2S2C high-QPS probe):

- At targetQps around `8000`, the RabbitMQ node (`t3.small`) becomes the primary bottleneck.
- Observed behavior: broker CPU approaches saturation, queue/rate curves show periodic spikes, and forwarding cannot keep up with ingress.

- Interpretation: the small instance class is under-provisioned for this QPS range; burstable credit and runtime pause effects (including GC/scheduler jitter) amplify spike behavior.
- Recommendation for subsequent 4S4C runs: upgrade RabbitMQ instance size first (e.g., `t3.medium` or larger) before attributing bottlenecks to server/consumer logic.

3.8 Post-Assignment Architecture Direction

For production-oriented evolution (beyond assignment scope), a better direction is:

- co-locate lightweight consumer/broadcast workers with each server node to reduce cross-node fanout overhead
- use a partitioned message log (or partitioned queue topology) keyed by `roomId` for predictable ordering and parallelism
- add a dedicated `presence service` to track `user -> server` / `room -> server set` mappings
- prefer local in-memory copy/broadcast whenever sender and target sessions are on the same server, and only do cross-node forwarding when required by presence lookup

This design reduces unnecessary global fanout traffic and improves scale efficiency while keeping room-level ordering manageable.